Technical Notes: Verilog RTL and RISC-V Practice Activities

Day 4: Pipeline hazards, analysis and submission

4.1 Data hazard handling

- **Approach:** Forwarding from EX/MEM and MEM/WB into the EX stage, plus a one-cycle stall when a load is followed by a dependent instruction.
- **Verification:** A hazard program with back-to-back dependent instructions and a load-use case; the testbench checks the final register values.
- **Result:** Correct results (x3 = 12 for the RAW case, x3 = 43 for the load-use case).
- **Issues:** No significant issues recorded.

4.2 Control hazard handling

- **Approach:** When a branch or jump is taken, the wrong-path instructions in IF and ID are flushed.
- **Verification:** A program with taken branches, not-taken branches and a jump; the waveform shows the flush.
- **Result:** The wrong-path write does not happen (x5 stays 0) and the target instruction completes.
- **Issues:** No significant issues recorded.

4.3 Performance comparison

- **Approach:** Ran programs 3.2(a) and 3.2(c) on both CPUs and recorded cycles, instructions and CPI.
- **Verification:** Both CPUs give the same results.
- **Result:** Sum 1..10: single-cycle 34 cycles (CPI 1.030), pipeline 56 cycles (CPI 1.697). Array maximum: 58 cycles (1.018) versus 91 cycles (1.596).
- **Conclusion:** The pipeline is slower on these short, branch-heavy programs. Fill and drain overhead and flushes on taken branches dominate. Each flush event costs 2 bubble cycles, and the maximum program adds 7 load-use stalls. Longer straight-line programs would amortize that overhead, so the pipeline's advantage is a higher possible clock frequency, not a lower CPI.
- **Issues:** Instruction counts exclude the ECALL stop marker, so CPI is cycles divided by non-ECALL instructions.

4.4 Git submission

- **Approach:** One repository with day1/ to day4/ (each with src/, tb/, waves/), report/, and README.md.
- **Verification:** Not yet done. After pushing, clone the repository into a new folder and re-run the simulations from the README.
- **Result:** Not complete until the GitHub link is added and the fresh-clone check has been run. Repository link: to be added after the push.
- **Issues:** None recorded.

4.5 Hazard waveform report

- **Approach:** Three hazards (consecutive RAW, load-use, taken branch), each run once with the fix disabled and once enabled.
- **Verification:** hazard_tb checks that the broken runs fail and the corrected runs pass.
- **Result:** Without the fix: x3 = 5 instead of 12, x3 = 1 instead of 43, and a wrong-path x5 write of 7. With the fix: all correct.
- **Issues:** None in the design. Status: the six before/after screenshots still have to be captured from hazards.vcd in the ModelSim GUI.

4.6 Performance counters

- **Approach:** Counters for total cycles, instructions retired, stall cycles and flush events added to the pipeline.
- **Verification:** Values printed at the end of each program in the testbench.
- **Result:** For example, bubble sort: 358 cycles, 28 stalls, 39 flush events.
- **Issues:** No significant issues recorded.

4.7 Extended program runs

- **Approach:** Ran bubble sort, GCD and shift-and-add on the pipeline and compared with the single-cycle CPU.
- **Verification:** Results match the single-cycle CPU.
- **Result:** Table now covers six programs. CPI ranges from 1.311 (Fibonacci) to 1.895 (GCD).
- **Issues:** No significant issues recorded.

4.8 Architecture diagram and demo

- **Approach:** Block diagram of the five stages, four pipeline registers, forwarding paths and the stall and flush signals. Demo notes follow the five-step plan in day4_demo.md.
- **Verification:** Compare the diagram against the RTL signal names before submitting.
- **Result:** The diagram is the SVG file pipeline_architecture.svg.
- **Issues:** None recorded.